

Recap Programmazione 2 - Lezioni e Laboratori

Appunti del Corso

Marzo 2026

Indice

1	Recap di Programmazione 1	3
1.1	Dal "Small" al "Large"	3
1.2	La Notional Machine	3
1.3	Gestione della Memoria e Variabili	3
1.3.1	Visibilità e Shadowing	3
1.3.2	Lo Stack e i Frame	3
1.3.3	Puntatori e Memoria Dinamica	3
2	Introduzione al Paradigma Object-Oriented	4
2.1	I Pilastri dell'Object-Oriented	4
2.2	Terminologia Essenziale	4
2.3	Il Linguaggio Java	5
2.4	Esempio di codice: Modellare un blocco (TNT)	5
3	Organizzazione, Visibilità e Stato Statico	5
3.1	Package e Struttura del Progetto	5
3.2	Modificatori di Accesso (Incapsulamento)	6
3.3	Membri Statici vs Membri d'Istanza	6
3.4	Il Modificatore Final e il Pattern Singleton	6
4	Incapsulamento, Invarianti e Gestione Memoria	6
4.1	Incapsulamento e Invarianti	7
4.2	Passaggio dei Valori in Java	7
4.3	Layout della Memoria	7
5	Ereditarietà e Gerarchie di Tipi	7
5.1	Superclassi e Sottoclassi	8
5.2	Generalizzazione e Specializzazione	8
5.3	Il Problema del Diamante	8
6	Classi Astratte, Overloading e Overriding	8
6.1	Classi Astratte	8
6.2	Overloading (Sovraccarico)	9
6.3	Overriding (Sovrascrittura)	9
6.4	Enums nelle Classi	9

7	Interfacce, Polimorfismo e Metodi Default	10
7.1	Definizione e Contratto	10
7.2	Ereditarietà Multipla e Flessibilità	10
7.3	Estensione tra Interfacce e Metodi Default	10
8	Ereditarietà vs Composizione e lo Strategy Pattern	11
8.1	Limiti dell'Ereditarietà e Vantaggi della Composizione	11
8.2	Lo Strategy Pattern	11
8.3	Applicazione Pratica: Il Piglin	12

1 Recap di Programmazione 1

La prima lezione funge da ponte tra le conoscenze di base e la programmazione di sistemi complessi.

1.1 Dal "Small" al "Large"

Il corso segna il passaggio fondamentale tra due approcci:

- **Programming in the Small:** Si concentra su piccole procedure e sulla familiarizzazione con la sintassi del codice.
- **Programming in the Large:** Focus sulla costruzione di sistemi complessi che richiedono **divide et impera** (suddivisione del lavoro), **manutenibilità** (modificabilità nel tempo) e **robustezza** agli errori.

Il supporto principale per gestire la programmazione "in the large" è fornito dai **linguaggi orientati agli oggetti (OO)**.

1.2 La Notional Machine

Un concetto pedagogico chiave è la **Notional Machine**. Essa rappresenta l'astrazione mentale che il programmatore costruisce per comprendere la **semantica** del linguaggio, ovvero come il runtime gestisce l'esecuzione del codice.

1.3 Gestione della Memoria e Variabili

1.3.1 Visibilità e Shadowing

Le variabili hanno diversi livelli di visibilità:

- **Globali:** Dichiarate fuori dalle funzioni, accessibili ovunque (es. `int vg;`).
- **Locali:** Visibili solo nel blocco in cui sono dichiarate.
- **Shadowing:** Avviene quando una variabile locale ha lo stesso nome di una globale, rendendo quest'ultima temporaneamente irraggiungibile in quel contesto.

1.3.2 Lo Stack e i Frame

L'allocazione per le funzioni avviene tramite gli **allocation records** o stack frames.

- Ogni chiamata a funzione crea un frame nello stack contenente parametri e variabili locali.
- Nelle funzioni **ricorsive** (es. il calcolo del fattoriale), ogni chiamata genera un nuovo frame, permettendo di mantenere stati distinti per ogni livello di ricorsione.

1.3.3 Puntatori e Memoria Dinamica

I puntatori manipolano direttamente gli indirizzi di memoria.

- **Passaggio per puntatore:** Permette a una funzione di modificare una variabile definita in un altro frame perché l'originale è ancora "viva" sullo stack.

- **Allocazione Dinamica:** Avviene nell'**heap** tramite **new** e richiede la de-allocazione manuale con **delete** per evitare memory leak.

```
void test_mem_dyn() {
    int* px;
    px = new int; // Allocazione dinamica nell'heap
    *px = 3;      // De-referenziazione
    delete(px);  // De-allocazione obbligatoria
}
```

Listing 1: Esempio di allocazione dinamica (test_memoria.cpp)

2 Introduzione al Paradigma Object-Oriented

Il passaggio al Programming in the Large richiede un approccio modulare per gestire l'evoluzione del codice e il lavoro di gruppo. Il paradigma orientato agli oggetti (OO) fornisce gli strumenti necessari per massimizzare il ****riuso del codice**** e minimizzare gli errori derivanti dalle modifiche nel tempo.

2.1 I Pilastri dell'Object-Oriented

Il paradigma si basa su cinque concetti fondamentali:

- **Oggetti:** Entità dotate di uno **stato** (dati), un **comportamento** (metodi), un identificativo univoco e una locazione in memoria.
- **Incapsulamento:** Protezione dello stato interno dell'oggetto. Solo ciò che è strettamente necessario viene esposto all'esterno tramite interfacce (*information hiding*).
- **Classificazione:** Gli oggetti vengono organizzati in classi, permettendo di raggruppare entità con caratteristiche comuni.
- **Specializzazione:** Attraverso l'ereditarietà, è possibile estendere classi esistenti aggiungendo nuovi stati o comportamenti specifici.
- **Polimorfismo:** La capacità di un oggetto di essere utilizzato in contesti di tipo differente, favorendo la flessibilità del codice.

2.2 Terminologia Essenziale

Per comprendere la programmazione ad oggetti è necessario definire con precisione alcuni termini tecnici:

- **Classe:** Rappresenta il "blueprint" o lo stampino (es. lo stampino per i biscotti). Definisce la forma dello stato (*memory footprint*) e il comportamento comune.
- **Oggetto:** È l'istanza specifica di una classe (es. il singolo biscotto decorato). Ogni oggetto ha il proprio stato che può variare indipendentemente dagli altri.
- **Metodo:** Un comportamento definito nella classe e invocabile sull'oggetto.
- **Campo (Variabile d'istanza):** Una componente dello stato interno di un oggetto.
- **Firma (Signature):** La definizione dei tipi di input e dell'output di un metodo.

2.3 Il Linguaggio Java

Java è un linguaggio fortemente tipato e modulare. Una caratteristica distintiva è che tutto deve risiedere all'interno di una classe; non esistono funzioni o variabili "sciolte".

- **Metodo main:** È il punto di ingresso di ogni applicazione Java. La sua firma fissa è `public static void main(String[] args)`.
- **Tipi di dati:** Java distingue tra **Tipi Primitivi** (come `int`, `boolean`, `char`) e tipi definiti dall'utente (Classi), usati per modellare domini complessi come quello di Minecraft.
- **Enums:** Tipi speciali utilizzati per definire un set fisso di costanti, come le direzioni (`NORTH`, `SOUTH`, ecc.) o i livelli degli strumenti (`ToolTier`).

2.4 Esempio di codice: Modellare un blocco (TNT)

L'esempio della classe TNT mostra come si evolve la definizione di un oggetto, partendo da una semplice etichetta fino alla definizione di uno stato interno complesso.

```
// Versione base: definisce solo il tipo
public class TNT { }

// Versione con stato: aggiunta di campi d'istanza
public class TNT {
    public int fuseLength = 10;      // Tempo di innesco
    public double explosionPower;    // Potenza esplosione
    public boolean isIgnited;        // Stato della miccia
}
```

Listing 2: Evoluzione della classe TNT in Java

3 Organizzazione, Visibilità e Stato Statico

Questa sezione approfondisce la strutturazione di progetti complessi e le regole che governano l'accesso ai dati e la condivisione dello stato tra gli oggetti.

3.1 Package e Struttura del Progetto

I **Package** sono strumenti semantici utilizzati per organizzare classi correlate in una gerarchia logica (es. `lecture03.packages.blocks`). Essi permettono di:

- Fare ordine in progetti di grandi dimensioni.
- Evitare conflitti di nomi tra classi diverse.
- Definire il **nome completo** di una classe, che include il percorso del suo package (es. `lecture03.packages.entities.Player`).

3.2 Modificatori di Accesso (Incapsulamento)

L'incapsulamento si avvale dei modificatori di accesso per regolare la visibilità di classi, metodi e campi:

- **public**: Il membro è accessibile da qualsiasi altra classe nel progetto.
- **private**: Il membro è visibile solo all'interno della classe in cui è dichiarato. È fondamentale per proteggere l'integrità dei dati (ad esempio, la salute di un giocatore non dovrebbe essere modificabile liberamente dall'esterno).
- **Differenza Java/C++**: In Java, se non specificato, l'accesso è "package-private" (visibile solo nel package). In C++, i membri di una classe sono **privati di default**.

3.3 Membri Statici vs Membri d'Istanza

La distinzione tra ciò che appartiene all'oggetto e ciò che appartiene alla classe è cruciale:

- **Campi d'istanza**: Ogni oggetto ha la sua copia personale. Se abbiamo più istanze di TNT, innescarne una (`isIgnited = true`) non influisce sulle altre.
- **Campi statici**: Appartengono alla classe stessa e sono condivisi da tutte le istanze. Sono utili per costanti globali del gioco o per dati che devono essere univoci per l'intera esecuzione (es. `GameConstants`).

3.4 Il Modificatore Final e il Pattern Singleton

- **final**: Una variabile dichiarata `final` può essere inizializzata una sola volta e il suo valore non può più essere cambiato (immutabilità). Si usa spesso per attributi che definiscono l'identità, come il nome di un giocatore.
- **Singleton Pattern**: Si applica quando una classe deve avere **una sola istanza** in tutto il sistema. Un esempio tipico in Minecraft è il `DragonEgg`, di cui esiste un solo esemplare nel mondo di gioco.

```
public class GameConstants {
    public static final double GRAVITY = 9.81; // Costante
        globale condivisa
}

public class FinalPlayer {
    public final String name; // Inizializzato una volta, mai piu
        modificato
    public FinalPlayer(String name) { this.name = name; }
}
```

Listing 3: Esempio di campi statici e final

4 Incapsulamento, Invarianti e Gestione Memoria

Questa sezione approfondisce le tecniche per proteggere l'**integrità dei dati** e le modalità con cui il runtime gestisce gli oggetti.

4.1 Incapsulamento e Invarianti

L'obiettivo centrale dell'incapsulamento non è semplicemente l'occultamento delle informazioni, ma la garanzia del rispetto delle **invarianti**. Una **invariante** è una condizione logica che deve rimanere costantemente vera durante l'esistenza di un oggetto; ad un **Player**, per citare un caso, non deve essere consentito avere un valore di salute negativo. Rendendo i campi **private**, si obbliga l'accesso tramite metodi che effettuano la **validazione dei dati**, impedendo il passaggio a stati del sistema inconsistenti.

4.2 Passaggio dei Valori in Java

Il comportamento del linguaggio durante l'invocazione dei metodi varia in base alla natura del dato trattato:

- **Tipi Primitivi:** Il sistema effettua una **copia del valore**. Qualsiasi modifica eseguita sul parametro non ha alcun impatto sulla variabile originale.
- **Oggetti:** Il sistema effettua una **copia del riferimento**, ovvero dell'indirizzo di memoria. Di conseguenza, le alterazioni allo stato dell'oggetto compiute nel metodo sono visibili ovunque, poiché l'oggetto è il medesimo, pur restando impossibile far puntare il riferimento di partenza a un'entità differente.

4.3 Layout della Memoria

La distinzione tra la definizione strutturale e l'occupazione fisica è fondamentale:

- **Class Memory Layout:** Rappresenta lo schema con cui la classe organizza la gerarchia e la disposizione dei dati.
- **Object Memory Layout:** Indica lo spazio concreto che una specifica istanza occupa nell'**Heap** durante l'esecuzione.

```
public class Player {  
    private int health = 20;  
  
    public void setHealth(int h) {  
        if (h >= 0 && h <= 20) { // Controllo della validità  
            this.health = h;  
        }  
    }  
}
```

Listing 4: Validazione dello stato interno

5 Ereditarietà e Gerarchie di Tipi

L'ereditarietà è introdotta come uno strumento fondamentale per favorire la modularizzazione e il riuso del codice. Attraverso la keyword **extends**, è possibile definire una relazione di tipo **is-a**, stabilendo che un oggetto di una sottoclasse è a tutti gli effetti un tipo della sua superclasse.

5.1 Superclassi e Sottoclassi

La **Superclasse** (come `Entity`) funge da contenitore per i campi e i metodi comuni a tutte le entità del sistema, quali la posizione o la salute. Le **Sottoclassi** (come `Zombie` o `Creeper`) estendono la superclasse ereditandone le caratteristiche e aggiungendo comportamenti specifici, come la capacità esplosiva del `Creeper`.

5.2 Generalizzazione e Specializzazione

La **Generalizzazione** consiste nel fattorizzare il codice comune verso l'alto nella gerarchia per evitare inutili duplicazioni. La **Specializzazione** permette invece a tipi differenti di rispondere in modo specifico a determinati stimoli, pur essendo gestiti tramite tipi generici nel resto del sistema.

5.3 Il Problema del Diamante

Il **Diamond Problem** rappresenta un'ambiguità strutturale che occorre nell'ereditarietà multipla, tipicamente quando una classe eredita da due genitori che condividono un antenato comune. Per prevenire conflitti e garantire la semplicità del runtime, Java **non consente l'ereditarietà multipla tra classi**, permettendo a una classe di estendere un'unica superclasse.

```
// Definizione della superclasse comune
public class Entity {
    public int health;
}

// Estensione per creare un tipo specifico
public class Zombie extends Entity {
    public void groan() {
        System.out.println("Groan...");
    }
}
```

Listing 5: Applicazione dell'ereditarietà tra classi

6 Classi Astratte, Overloading e Overriding

Questa sezione analizza le tecniche avanzate per raffinare le gerarchie di classi, permettendo di gestire comportamenti specifici e modelli di dati più complessi attraverso il sistema di tipi di Java.

6.1 Classi Astratte

Le classi **astratte**, identificate dalla keyword `abstract`, fungono da modelli parziali per altre classi. Esse presentano le seguenti caratteristiche:

- **Non istanziabili:** Non è consentito creare un oggetto direttamente da una classe astratta. Risulta impossibile creare un `Block` generico, ma è necessario riferirsi a tipi specifici come `Dirt`.

- **Finalità:** Definiscono uno stato e un comportamento comune a tutte le sottoclassi, imponendo una **struttura coerente** all'intero sistema (come nel caso delle classi Block ed Entity).
- **Metodi Astratti:** Consentono di dichiarare la **firma** di un metodo senza fornirne l'implementazione, obbligando legalmente le sottoclassi a definirne il comportamento specifico.

6.2 Overloading (Sovraccarico)

L'**overloading** si verifica quando una classe contiene più metodi con il medesimo nome ma **firme differenti**. La firma viene considerata diversa se cambia il numero, il tipo o l'ordine dei parametri. Una classe come Pick (piccone) utilizza questa tecnica per gestire l'interazione con oggetti di natura differente, disponendo di metodi distinti per trattare Chicken, Dirt o Bedrock.

6.3 Overriding (Sovrascrittura)

L'**overriding** avviene quando una sottoclasse fornisce una nuova implementazione di un metodo già definito nella sua superclasse.

- **Scopo:** Permettere a un oggetto di rispondere in modo **specializzato** a un'invocazione (si consideri il comportamento differente di Andesite o GoldBlock quando vengono colpiti).
- **Annotazione @Override:** In Java, viene utilizzata per segnalare esplicitamente la sovrascrittura, permettendo al compilatore di rilevare errori nelle firme dei metodi.

6.4 Enums nelle Classi

Le classi possono includere al proprio interno dei **tipi enumerati (Enums)** per definire set di costanti strettamente legati alla classe stessa. Una configurazione frequente vede l'utilizzo di Block.Material per elencare varianti quali DIRT o BEDROCK, garantendo la **sicurezza dei tipi** ed evitando errori di digitazione.

```
// Definizione di un modello parziale
public abstract class Block {
    public enum Material { DIRT, GOLD_BLOCK }

    // Firma senza implementazione
    public abstract void onBreak();
}

// Sottoclasse che fornisce l'implementazione specifica
public class Dirt extends Block {
    @Override
    public void onBreak() {
        System.out.println("Il blocco di terra è stato rimosso");
    }
}
```

Listing 6: Implementazione di classi astratte e sovrascrittura

7 Interfacce, Polimorfismo e Metodi Default

Le interfacce rappresentano un'evoluzione del concetto di astrazione, permettendo di definire un **contratto** che le classi devono rispettare senza imporre una gerarchia rigida di ereditarietà.

7.1 Definizione e Contratto

A differenza delle classi, le interfacce non possono contenere uno stato (campi d'istanza). Esse definiscono esclusivamente **comportamenti** attraverso firme di metodi. Una classe che implementa un'interfaccia si impegna a fornire un'implementazione concreta per tutti i metodi dichiarati, garantendo una **struttura coerente** per entità diverse.

7.2 Ereditarietà Multipla e Flessibilità

Mentre Java vieta l'ereditarietà multipla tra classi per evitare il problema del diamante, consente a una singola classe di implementare **molteplici interfacce**. Questa caratteristica permette a un'entità di assumere diversi ruoli contemporaneamente all'interno del sistema. Un **Skeleton**, ad un tempo, può essere gestito come un generico **Enemy** o come un **RangedAttackMob**, a seconda delle necessità del runtime. Tale approccio potenzia drasticamente il polimorfismo, poiché il codice può interagire con oggetti eterogenei basandosi esclusivamente sulle capacità comuni definite dalle loro interfacce.

7.3 Estensione tra Interfacce e Metodi Default

Le interfacce possono estendersi tra loro proprio come le classi. Una interfaccia specializzata come **Boss** può estendere **Enemy**, aggiungendo requisiti specifici per le entità di alto livello pur mantenendo la compatibilità con i tipi base. Inoltre, l'introduzione dei **metodi default** permette di inserire un'implementazione di base direttamente all'interno dell'interfaccia, facilitando l'evoluzione del codice e fornendo comportamenti predefiniti che le classi possono scegliere di utilizzare o sovrascrivere.

```
// Definizione di un comportamento d'attacco
public interface Enemy {
    void playAggressiveSound();
    void attack(Entity target);
}

// Estensione tra interfacce
public interface Boss extends Enemy {
    // Metodi specifici per i Boss
}

// Classe che implementa piu ruoli (Skeleton)
public class Skeleton extends Entity implements Enemy,
    RangedAttackMob {
    @Override
    public void playAggressiveSound() {
        System.out.println("Skeleton: ⚔️*Bone⚔️Rattle*");
    }

    @Override
    public void attack(Entity target) {
        // Implementazione specifica dell'attacco
    }
}
```

Listing 7: Implementazione di interfacce multiple e gerarchie

8 Ereditarietà vs Composizione e lo Strategy Pattern

Il design del software richiede spesso di scegliere tra l'estensione delle classi e l'aggregazione di componenti. Questa sezione esplora come la **composizione** possa superare i limiti di una gerarchia rigida.

8.1 Limiti dell'Ereditarietà e Vantaggi della Composizione

L'ereditarietà crea un legame forte tra superclasse e sottoclasse. Se un'entità deve cambiare comportamento durante l'esecuzione, l'ereditarietà risulta inadeguata poiché il tipo di un oggetto è fissato al momento della creazione. La **composizione** segue il principio secondo cui un oggetto "ha un" comportamento invece di "essere un" comportamento. Questo approccio favorisce il **disaccoppiamento**, rendendo il sistema più flessibile e facilitando il riuso di componenti logiche isolate.

8.2 Lo Strategy Pattern

Lo **Strategy Pattern** è un design pattern comportamentale che permette di definire una famiglia di algoritmi, incapsularli e renderli intercambiabili. Invece di implementare un comportamento direttamente nella classe, l'oggetto delega l'esecuzione a un'interfaccia specifica. Questo permette di:

- Cambiare l'algoritmo a **runtime**.

- Isolare la logica complessa dal contesto principale.
- Evitare lunghe sequenze di istruzioni condizionali (if-else o switch) per gestire varianti diverse.

8.3 Applicazione Pratica: Il Piglin

Nel modello di Minecraft, un **Piglin** può attaccare in modi diversi (usando i pugni o una balestra). Utilizzando lo Strategy Pattern, la classe non ha bisogno di conoscere i dettagli dell'attacco, ma interagisce solo con l'interfaccia **AttackStrategy**.

```
// Interfaccia che definisce il comportamento generico
public interface AttackStrategy {
    void execute();
}

// Implementazione specifica per l'attacco a distanza
public class CrossbowAttack implements AttackStrategy {
    @Override
    public void execute() {
        System.out.println(">>_Attacco:_*Scoccata_freccia*");
    }
}

// Classe contesto che utilizza la composizione
public class Piglin {
    private AttackStrategy currentStrategy;

    public void setStrategy(AttackStrategy strategy) {
        this.currentStrategy = strategy;
    }

    public void performAttack() {
        currentStrategy.execute();
    }
}
```

Listing 8: Implementazione dello Strategy Pattern